

may need to do some of the following things to make your Web server accessible by others.

What to do

```
cd directory/to/serve
python -m SimpleHTTPServer 8080
```

That's it! This just took less than 5 minutes. Anyone can now point a browser at <http://your.machine:8080/> to access the tree under the directory-to-serve. If that directory has a file called *index.html*, that's what will be served. If not, you'll see a list of files in the directory.

The number (8080) is the port on which the server listens. The nice thing is that for ports above 1024, you don't need super user privileges, but can run the server as an ordinary user.

Of course, Python must be installed on the platform. But Python comes with many platforms these days; perhaps not with Arch Linux. And, as far as I can ascertain, with Python comes *SimpleHTTPServer*. The *-m* tells Python to look for *SimpleHTTPServer* in its default search path, which means that the command should work no matter which version of Python has been installed.

Wrinkles

I can't help myself. I found I needed to tidy this up a little. When invoked as above, *SimpleHTTPServer* logs every attempt to access the server. These log entries go to *stdout*. My first variation was:

```
python -m SimpleHTTPServer 8080 > /tmp/log &
```

For most people, this is more than enough. However, I decided that what would be even better would be a script to invoke the server. Here it is:

```
#!/bin/sh
#       SimpleHTTPServer.sh - start the Python SimpleHTTPServer

mydir=`dirname $0`           # directory this file is run from
cd $mydir

[ -s webserver_thisrun.log ] && {
    cat webserver_thisrun.log >> webserver.log
    rm webserver_thisrun.log
}

{
cat <<EOF
-----
EOF
date
```

```
python -m SimpleHTTPServer 8080
) >> webserver_thisrun.log 2>&1 &
```

```
sleep 1
while [ ! -s webserver_thisrun.log ]
do
    sleep 1
done
cat webserver_thisrun.log
```

I wrote this script with the intention that it be located in the directory above the directory-to-be-served. It maintains two log files: a long-term log of all runs, *webserver.log*, and a log of the current invocation, *webserver_thisrun.log*. When run, the script manages some housekeeping with log files; it then invokes *SimpleHTTPServer*, redirecting output to the current log file. The last few lines are intended to catch infant mortality—like if the port is already in use.

I created a sub-directory, rather unimaginatively called *www*, to hold the content I wanted served. I then emailed my boss to visit <http://my.machine:8080/www/>.

Unsurprisingly, as my sojourn at Acme Corporation continued, the content on the website grew and grew, all effortlessly served by *SimpleHTTPServer*. This set-up ran quite happily for over two months at Acme Corporation. For all I know, it's still running. I feel sure that you too will get many years of happy usage from *SimpleHTTPServer*.

The pros and cons

I don't think of *SimpleHTTPServer* as a replacement for Apache; it won't do CGI (but a related script, *CGIHTTPServer.py*, looks like it does). If you have heavy-duty Web-serving to do, then Apache is probably the answer. But if you just want something that's quick and easy to set up, which requires no special privileges and has an extremely light footprint, I think it would be hard to surpass *SimpleHTTPServer*. It has such a light footprint that you could even set up several separate servers on a single old-and-slow machine, and still get acceptable performance.

Notes

While researching this article, I've learnt that *SimpleHTTPServer* has been around for at least nine years, and was available as far back as Red Hat 8 and Python 2.2.1 or probably even earlier.

SimpleHTTPServer is built on *BaseHTTPServer*, which accepts a single optional argument. In the absence of an argument, it will listen on port 8000; otherwise, it will use the port number supplied. In the standard code, there is no ability to specify the address on which it will listen; it listens on all interfaces. It's a bit of a shame, really. If you want to be able to control that, you are going to have to write some code. This link, <http://bit.ly/4yzEZF>, provides an example.